POLY PMNA · DIPLOMA CURRICULUM REVISION 2026

Course 3051 | Semester III | Theory | 4 Credits

Computer Programming

A complete syllabus-aligned handbook with clear explanations, practical or exam guidance, Malayalam-support notes, diagrams, activities and revision tools.

Programmes	**Electronics Engineering**
Contact hours	**60**
Teaching scheme	**3:1:0:0**
Credits	**4**
CIE / ESE	**Theory CIE 40 / Theory ESE 60**
Self-learning	**5.5 h/week**

CURRICULUM INTEGRITY

Official Source Declaration and Verified Inconsistencies

The attached Revision 2026 index identifies Course 3051 as Computer Programming in Semester III for Electronics Engineering. The official SITTTR detailed source was unavailable during this cycle. With user approval, explanatory coverage uses a public diploma Programming in C curriculum and a current C learning outline; it is not represented as recovered official SITTTR Course 3051 detailed-syllabus wording.

Source treatment: Teaching scheme, credits and assessment entries are provisional placeholders. The four modules are a transparent C-programming study structure based on the cited public sources and must be reconciled with restored official Course 3051 material.

Verified source note: The official SITTTR detailed course source was inaccessible. Public sources used for algorithms, C syntax, control flow, derived data types and functions/pointers are linked in References.

COURSE DASHBOARD

What You Are Expected to Learn

60
Contact hours

4
Credits

Theory CIE 40
CIE

Theory ESE 60
ESE

Course introduction

Computer Programming introduces problem formulation and the C language for engineering tasks. Students learn to translate algorithms into safe, readable programs using expressions, control flow, arrays, functions, pointers and structured data.

> **Malayalam support:** □ handbook □□□□□□□□□□□□ syllabus topic □□□□□ □□□□□□□□□; □□□□□□□□ principle, diagram/procedure, application, common mistake □□□□□□ □□□□□□□□ □□□□□□□□.

Prerequisite foundation

Basic computer use, arithmetic reasoning and willingness to practise programs in a compiler/IDE.

Use prerequisites only as a revision bridge. The current course outcomes and detailed syllabus define the required performance.

Teaching and assessment scheme

L	T	P	S	Self-learning/week	Contact hours	Credits	CIE	ESE
3	1	0	0	5.5 h/week	60	4	Theory CIE 40	Theory ESE 60

STUDY METHOD

How to Use This Handbook

1 · Understand

Read terms, conditions and purpose; make a short definition in your own words.

2 · Visualise

Follow the diagram, circuit, flow or procedure and label it accurately.

3 · Apply

Use the method block, calculation or practical checklist without skipping units or safety checks.

4 · Revise

Use questions, quick cards and common-mistake notes before examination.

OFFICIAL STATEMENTS

Objectives and Course Outcomes

Official course objectives

1. Formulate problems using algorithms, flowcharts and pseudocode.
2. Write, compile, run and debug elementary C programs.
3. Apply operators, selection and iteration to solve logical problems.
4. Use arrays, strings, structures, functions and pointers in small programs.

Student-friendly meaning

You should be able to recognise the relevant system or material, explain its principle, communicate through a correct diagram/table where needed and follow a defensible solution or practical method.

> **Malayalam support:** CO □□□□□□ exam-□ □□□□□ □□□□□□□ □□□□□□ □□□□□ □□□□□□□□.
> Define, explain, apply □□□□□ action words □□□□□□□□□□□□□.

Official Course Outcomes

CO	Official outcome	Bloom level
CO1	Formulate a simple problem as an algorithm, flowchart or pseudocode.	Apply
CO2	Write and debug C programs using data types, expressions and input/output.	Apply
CO3	Use selection and loop statements to implement logical program flow.	Apply
CO4	Use arrays, strings, structures, functions and pointers in appropriate small programs.	Apply

Official CO–PO articulation matrix

CO	PO1	PO2	PO3	PO4	PO5	PO6	PO7–PO11
CO1	3	2	2	2	2	–	–
CO2	3	2	2	2	2	–	–
CO3	3	2	2	2	2	–	–
CO4	3	2	2	2	2	–	–

SYLLABUS COVERAGE

Curriculum Coverage Map

All official major topics mapped to handbook chapters

Module	Title	Official major topics	Hours	CO	Handbook section
1	Problem Solving and C Foundations	Problem analysis, algorithms, flowcharts, pseudocode, translators, C program structure, tokens, variables, types and input/output.	Approx. 15 hours	CO1	Module 1
2	Expressions, Decisions and Loops	Arithmetic/relational/logical/bitwise operators, precedence, type conversion, if/switch and for/while/do-while loops.	Approx. 15 hours	CO2	Module 2
3	Arrays, Strings and Structured Data	One-/two-dimensional arrays, strings, indexing, standard string operations, structures and enumerations.	Approx. 15 hours	CO3	Module 3
4	Functions, Pointers and Program Design	Library/user functions, scope, parameters, return values, call by value/address, recursion, storage classes and pointers.	Approx. 15 hours	CO4	Module 4

LEARNING ROADMAP

How the Modules Connect

Problem Solving and C Foundations

Problem analysis, algorithms, flowcharts, pseudocode, translators, C program structure, tokens, variables, types and input/output.

CO1 · Approx. 15 hours

Expressions, Decisions and Loops

Arithmetic/relational/logical/bitwise operators, precedence, type conversion, if/switch and for/while/do-while loops.

CO2 · Approx. 15 hours

Arrays, Strings and Structured Data

One-/two-dimensional arrays, strings, indexing, standard string operations, structures and enumerations.

CO3 · Approx. 15 hours

Functions, Pointers and Program Design

Library/user functions, scope, parameters, return values, call by value/address, recursion, storage classes and pointers.

CO4 · Approx. 15 hours

MODULE 1

Problem Solving and C Foundations

Problem analysis, algorithms, flowcharts, pseudocode, translators, C program structure, tokens, variables, types and input/output.

Approx. 15 hours | CO1 | Understand · Apply

Learning goals

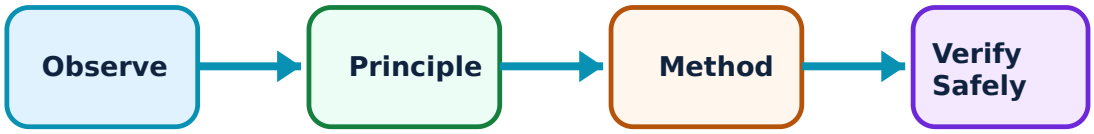
Identify core terms, explain the principle and complete the stated application or practical outcome.

Key evidence

Use a labelled diagram, table, graph, procedure or calculation that matches the topic.

Engineering habit

Check units, polarity/direction, ratings, materials and safety before concluding.



Study flow for Problem Solving and C Foundations: identify the system, state its principle, follow the correct method and verify the result safely.

1. Algorithms, flowcharts and pseudocode−

An algorithm is a finite, ordered solution plan. Flowcharts show control visually; pseudocode expresses the logic in structured natural-language style. All help detect logic errors before code is written.

Study and application method

State inputs, outputs, constraints and steps; trace one sample input by hand before converting the plan to C.

Connect the explanation to the official student learning outcome before attempting a numerical, diagram, procedure or viva answer.

Evidence of understanding

Use correct terminology, a labelled sketch/table where useful, correct units or conditions, and a conclusion linked to the observed or calculated result.

How to construct a complete answer

Given: identify the quantities, device condition, waveform or circuit arrangement.

Required: state exactly what must be found or explained.

Method: State inputs, outputs, constraints and steps; trace one sample input by hand before converting the plan to C.

Answer habit: show a labelled step, the relevant unit/condition and a short interpretation.

Malayalam support: □□□□□ concept □□□□□□□ □□□□□ □□□□□□□□□□□□□. □□□□ diagram / circuit / formula / procedure □□□□□□□□□□□ □□□□□□□□. □□□□□□□□□ result □□□□□□□□□□ application □□□□□□□ □□□□□□□□□□ □□□□□ □□□□□□□□□□□□□. Technical terms English-□ □□□□□ □□□□□□□□□□□□.

Common mistake / precaution: Do not start coding before defining what result the program must produce and how invalid input is handled.

2. C program structure and data−

A C program uses headers, a main function, declarations, statements and library calls. Variables have types that determine storage, range and permitted operations.

Study and application method

Write a minimal compilable program, declare variables before use, choose types intentionally and use readable names/comments.

Connect the explanation to the official student learning outcome before attempting a numerical, diagram, procedure or viva answer.

Evidence of understanding

Use correct terminology, a labelled sketch/table where useful, correct units or conditions, and a conclusion linked to the observed or calculated result.

How to construct a complete answer

Given: identify the quantities, device condition, waveform or circuit arrangement.

Required: state exactly what must be found or explained.

Method: Write a minimal compilable program, declare variables before use, choose types intentionally and use readable names/comments.

Answer habit: show a labelled step, the relevant unit/condition and a short interpretation.

Malayalam support: □□□□□ concept □□□□□□□ □□□□□ □□□□□□□□□□□□□□. □□□□ diagram / circuit / formula / procedure □□□□□□□□□□□ □□□□□□□□. □□□□□□□□□ result □□□□□□□□□□ application □□□□□□□ □□□□□□□□□□ □□□□□ □□□□□□□□□□□□□. Technical terms English-□ □□□□□ □□□□□□□□□□□□.

Common mistake / precaution: Compiler warnings are information, not decoration; investigate them before trusting output.

3. Input/output and formatted data−

Functions such as printf and scanf support interaction, but format specifiers must match variable types. Input validation and predictable output make programs testable.

Study and application method

Specify expected input, use correct format strings, check whether input succeeded where appropriate and print units/labels clearly.

Connect the explanation to the official student learning outcome before attempting a numerical, diagram, procedure or viva answer.

Evidence of understanding

Use correct terminology, a labelled sketch/table where useful, correct units or conditions, and a conclusion linked to the observed or calculated result.

How to construct a complete answer

Given: identify the quantities, device condition, waveform or circuit arrangement.

Required: state exactly what must be found or explained.

Method: Specify expected input, use correct format strings, check whether input succeeded where appropriate and print units/labels clearly.

Answer habit: show a labelled step, the relevant unit/condition and a short interpretation.

Malayalam support: □□□□□ concept □□□□□□□ □□□□□ □□□□□□□□□□□□□. □□□□ diagram / circuit / formula / procedure □□□□□□□□□□□ □□□□□□□□. □□□□□□□□□ result □□□□□□□□□□ application □□□□□□□ □□□□□□□□□□ □□□□□ □□□□□□□□□□□□□. Technical terms English-□ □□□□□ □□□□□□□□□□□□.

Common mistake / precaution: Never use unsafe or unbounded input patterns; protect array and buffer limits.

Module 1 revision cue: Re-read official major topics, redraw one key diagram/flow and explain one practical or numerical method without looking at notes.

MODULE 2

Expressions, Decisions and Loops

Arithmetic/relational/logical/bitwise operators, precedence, type conversion, if/switch and for/while/do-while loops.

Approx. 15 hours | CO2 | Understand · Apply

Learning goals

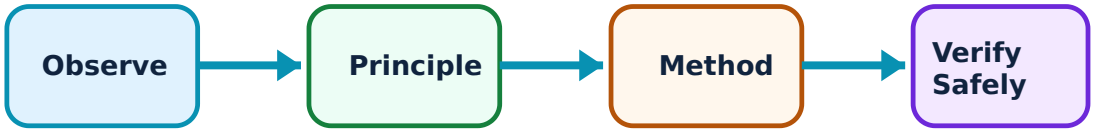
Identify core terms, explain the principle and complete the stated application or practical outcome.

Key evidence

Use a labelled diagram, table, graph, procedure or calculation that matches the topic.

Engineering habit

Check units, polarity/direction, ratings, materials and safety before concluding.



Study flow for Expressions, Decisions and Loops: identify the system, state its principle, follow the correct method and verify the result safely.

1. Operators and expressions−

Expressions combine operands and operators to compute a value. Precedence, associativity and type conversion affect evaluation and can produce unexpected results.

Study and application method

Parenthesise non-obvious expressions, use intermediate variables for clarity and test boundary values.

Connect the explanation to the official student learning outcome before attempting a numerical, diagram, procedure or viva answer.

Evidence of understanding

Use correct terminology, a labelled sketch/table where useful, correct units or conditions, and a conclusion linked to the observed or calculated result.

How to construct a complete answer

Given: identify the quantities, device condition, waveform or circuit arrangement.

Required: state exactly what must be found or explained.

Method: Parenthesise non-obvious expressions, use intermediate variables for clarity and test boundary values.

Answer habit: show a labelled step, the relevant unit/condition and a short interpretation.

Malayalam support: □□□□□ concept □□□□□□□ □□□□□ □□□□□□□□□□□□□□. □□□□ diagram / circuit / formula / procedure □□□□□□□□□□□ □□□□□□□□. □□□□□□□□□ result □□□□□□□□□□ application □□□□□□□ □□□□□□□□□□ □□□□□ □□□□□□□□□□□□□. Technical terms English-□ □□□□□ □□□□□□□□□□□□.

Common mistake / precaution: Do not confuse assignment = with equality comparison ==.

2. Selection statements−

if/else chooses between conditions; switch selects among discrete cases. Good conditions represent business/engineering rules clearly and include sensible default handling.

Study and application method

Write a truth table or decision table first, test each branch including boundary/default cases and keep nesting understandable.

Connect the explanation to the official student learning outcome before attempting a numerical, diagram, procedure or viva answer.

Evidence of understanding

Use correct terminology, a labelled sketch/table where useful, correct units or conditions, and a conclusion linked to the observed or calculated result.

How to construct a complete answer

Given: identify the quantities, device condition, waveform or circuit arrangement.

Required: state exactly what must be found or explained.

Method: Write a truth table or decision table first, test each branch including boundary/default cases and keep nesting understandable.

Answer habit: show a labelled step, the relevant unit/condition and a short interpretation.

Malayalam support: □□□□□ concept □□□□□□□ □□□□□ □□□□□□□□□□□□□. □□□□ diagram / circuit / formula / procedure □□□□□□□□□□□ □□□□□□□□. □□□□□□□□□ result □□□□□□□□□□ application □□□□□□□ □□□□□□□□□□ □□□□□ □□□□□□□□□□□□□. Technical terms English-□ □□□□□ □□□□□□□□□□□□.

Common mistake / precaution: A missing break in a switch may cause unintended fall-through unless it is deliberately documented.

3. Iteration−

for, while and do-while repeat work under different conditions. Correct initialisation, condition and update guarantee progress toward termination.

Study and application method

State loop invariant, initialise deliberately, trace first/last iteration and test zero/one/maximum-size cases.

Connect the explanation to the official student learning outcome before attempting a numerical, diagram, procedure or viva answer.

Evidence of understanding

Use correct terminology, a labelled sketch/table where useful, correct units or conditions, and a conclusion linked to the observed or calculated result.

How to construct a complete answer

Given: identify the quantities, device condition, waveform or circuit arrangement.

Required: state exactly what must be found or explained.

Method: State loop invariant, initialise deliberately, trace first/last iteration and test zero/one/maximum-size cases.

Answer habit: show a labelled step, the relevant unit/condition and a short interpretation.

Malayalam support: □□□□□ concept □□□□□□□ □□□□□ □□□□□□□□□□□□□. □□□□ diagram / circuit / formula / procedure □□□□□□□□□□□ □□□□□□□□. □□□□□□□□□ result □□□□□□□□□□ application □□□□□□□ □□□□□□□□□□ □□□□□ □□□□□□□□□□□□□. Technical terms English-□ □□□□□ □□□□□□□□□□□□.

Common mistake / precaution: Avoid accidental infinite loops and off-by-one indexing errors.

Module 2 revision cue: Re-read official major topics, redraw one key diagram/flow and explain one practical or numerical method without looking at notes.

MODULE 3

Arrays, Strings and Structured Data

One-/two-dimensional arrays, strings, indexing, standard string operations, structures and enumerations.

Approx. 15 hours | CO3 | Understand · Apply

Learning goals

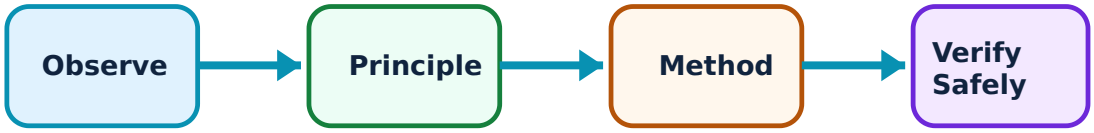
Identify core terms, explain the principle and complete the stated application or practical outcome.

Key evidence

Use a labelled diagram, table, graph, procedure or calculation that matches the topic.

Engineering habit

Check units, polarity/direction, ratings, materials and safety before concluding.



Study flow for Arrays, Strings and Structured Data: identify the system, state its principle, follow the correct method and verify the result safely.

1. Arrays and indexing−

An array stores same-type values in indexed order. C indexes start at zero, so valid element positions are determined by declared size.

Study and application method

Define size/meaning of each index, initialise before read, trace bounds and test first/last element cases.

Connect the explanation to the official student learning outcome before attempting a numerical, diagram, procedure or viva answer.

Evidence of understanding

Use correct terminology, a labelled sketch/table where useful, correct units or conditions, and a conclusion linked to the observed or calculated result.

How to construct a complete answer

Given: identify the quantities, device condition, waveform or circuit arrangement.

Required: state exactly what must be found or explained.

Method: Define size/meaning of each index, initialise before read, trace bounds and test first/last element cases.

Answer habit: show a labelled step, the relevant unit/condition and a short interpretation.

Malayalam support: □□□□□ concept □□□□□□□ □□□□□ □□□□□□□□□□□□□. □□□□ diagram / circuit / formula / procedure □□□□□□□□□□□ □□□□□□□□. □□□□□□□□□ result □□□□□□□□□□ application □□□□□□□ □□□□□□□□□□ □□□□□ □□□□□□□□□□□□□. Technical terms English-□ □□□□□ □□□□□□□□□□□□.

Common mistake / precaution: Out-of-bounds access is undefined behaviour and can silently corrupt a program.

2. Strings−

A C string is a character array terminated by a null character. Length, comparison and concatenation require capacity management and correct termination.

Study and application method

Allocate space including the terminator, state maximum input length and use bounded operations where available.

Connect the explanation to the official student learning outcome before attempting a numerical, diagram, procedure or viva answer.

Evidence of understanding

Use correct terminology, a labelled sketch/table where useful, correct units or conditions, and a conclusion linked to the observed or calculated result.

How to construct a complete answer

Given: identify the quantities, device condition, waveform or circuit arrangement.

Required: state exactly what must be found or explained.

Method: Allocate space including the terminator, state maximum input length and use bounded operations where available.

Answer habit: show a labelled step, the relevant unit/condition and a short interpretation.

Malayalam support: □□□□□ concept □□□□□□□ □□□□□ □□□□□□□□□□□□□. □□□□ diagram / circuit / formula / procedure □□□□□□□□□□□ □□□□□□□□. □□□□□□□□□ result □□□□□□□□□□ application □□□□□□□ □□□□□□□□□□ □□□□□ □□□□□□□□□□□□□. Technical terms English-□ □□□□□ □□□□□□□□□□□□.

Common mistake / precaution: Do not assume a character array is a valid string until it is correctly terminated.

3. Structures and enums−

Structures group related fields of different types; enumerations give meaningful names to a fixed set of integral states. They improve modelling and readability.

Study and application method

Design a record from the real-world entity, declare fields with appropriate types and access them through clear names.

Connect the explanation to the official student learning outcome before attempting a numerical, diagram, procedure or viva answer.

Evidence of understanding

Use correct terminology, a labelled sketch/table where useful, correct units or conditions, and a conclusion linked to the observed or calculated result.

How to construct a complete answer

Given: identify the quantities, device condition, waveform or circuit arrangement.

Required: state exactly what must be found or explained.

Method: Design a record from the real-world entity, declare fields with appropriate types and access them through clear names.

Answer habit: show a labelled step, the relevant unit/condition and a short interpretation.

Malayalam support: □□□□□ concept □□□□□□□ □□□□□ □□□□□□□□□□□□□. □□□□ diagram / circuit / formula / procedure □□□□□□□□□□□ □□□□□□□□. □□□□□□□□□ result □□□□□□□□□□ application □□□□□□□ □□□□□□□□□□ □□□□□ □□□□□□□□□□□□□. Technical terms English-□ □□□□□ □□□□□□□□□□□□.

Common mistake / precaution: Keep structure initialisation and field validation explicit; uninitialised fields create unreliable behaviour.

Module 3 revision cue: Re-read official major topics, redraw one key diagram/flow and explain one practical or numerical method without looking at notes.

MODULE 4

Functions, Pointers and Program Design

Library/user functions, scope, parameters, return values, call by value/address, recursion, storage classes and pointers.

Approx. 15 hours | CO4 | Understand · Apply

Learning goals

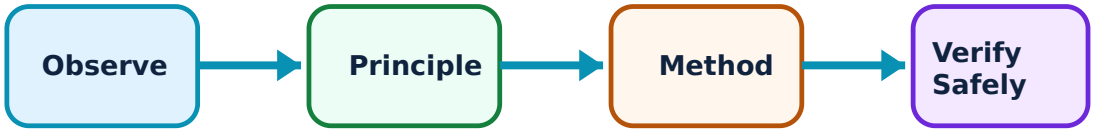
Identify core terms, explain the principle and complete the stated application or practical outcome.

Key evidence

Use a labelled diagram, table, graph, procedure or calculation that matches the topic.

Engineering habit

Check units, polarity/direction, ratings, materials and safety before concluding.



Study flow for Functions, Pointers and Program Design: identify the system, state its principle, follow the correct method and verify the result safely.

1. Functions and modular design−

Functions package a focused task with parameters and a result, reducing duplication and improving testability. Scope determines where variables can be used.

Study and application method

Write a prototype, specify input/output/side effects, test the function with normal and boundary cases and keep it single-purpose.

Connect the explanation to the official student learning outcome before attempting a numerical, diagram, procedure or viva answer.

Evidence of understanding

Use correct terminology, a labelled sketch/table where useful, correct units or conditions, and a conclusion linked to the observed or calculated result.

How to construct a complete answer

Given: identify the quantities, device condition, waveform or circuit arrangement.

Required: state exactly what must be found or explained.

Method: Write a prototype, specify input/output/side effects, test the function with normal and boundary cases and keep it single-purpose.

Answer habit: show a labelled step, the relevant unit/condition and a short interpretation.

Malayalam support: □□□□□ concept □□□□□□□ □□□□□ □□□□□□□□□□□□□. □□□□ diagram / circuit / formula / procedure □□□□□□□□□□□ □□□□□□□□. □□□□□□□□□ result □□□□□□□□□□ application □□□□□□□ □□□□□□□□□□ □□□□□ □□□□□□□□□□□□□. Technical terms English-□ □□□□□ □□□□□□□□□□□□.

Common mistake / precaution: Do not rely on hidden global state when parameters/return values express the design more clearly.

2. Pointers and addresses−

A pointer stores an address and allows indirect access to an object. Pointer use supports call-by-address, array traversal and dynamic structures but requires careful validity rules.

Study and application method

Draw memory boxes/arrows, initialise pointers, check validity before dereference and pass array length with array parameters.

Connect the explanation to the official student learning outcome before attempting a numerical, diagram, procedure or viva answer.

Evidence of understanding

Use correct terminology, a labelled sketch/table where useful, correct units or conditions, and a conclusion linked to the observed or calculated result.

How to construct a complete answer

Given: identify the quantities, device condition, waveform or circuit arrangement.

Required: state exactly what must be found or explained.

Method: Draw memory boxes/arrows, initialise pointers, check validity before dereference and pass array length with array parameters.

Answer habit: show a labelled step, the relevant unit/condition and a short interpretation.

Malayalam support: □□□□□ concept □□□□□□□ □□□□□ □□□□□□□□□□□□□. □□□□ diagram / circuit / formula / procedure □□□□□□□□□□□ □□□□□□□□. □□□□□□□□□ result □□□□□□□□□□ application □□□□□□□ □□□□□□□□□□ □□□□□ □□□□□□□□□□□□. Technical terms English-□ □□□□□ □□□□□□□□□□□.

Common mistake / precaution: Never dereference an uninitialised, null or dangling pointer.

3. Recursion and storage duration−

Recursion solves a problem by reducing it to smaller cases with a base case. Storage classes/scope/lifetime affect where variables exist and how long they retain values.

Study and application method

Define base case and reduction step, trace a small call stack and compare automatic/local versus static/global behaviour.

Connect the explanation to the official student learning outcome before attempting a numerical, diagram, procedure or viva answer.

Evidence of understanding

Use correct terminology, a labelled sketch/table where useful, correct units or conditions, and a conclusion linked to the observed or calculated result.

How to construct a complete answer

Given: identify the quantities, device condition, waveform or circuit arrangement.

Required: state exactly what must be found or explained.

Method: Define base case and reduction step, trace a small call stack and compare automatic/local versus static/global behaviour.

Answer habit: show a labelled step, the relevant unit/condition and a short interpretation.

Malayalam support: □□□□□ concept □□□□□□□ □□□□□ □□□□□□□□□□□□□. □□□□ diagram / circuit / formula / procedure □□□□□□□□□□□ □□□□□□□□. □□□□□□□□□ result □□□□□□□□□□ application □□□□□□□ □□□□□□□□□□ □□□□□ □□□□□□□□□□□□. Technical terms English-□ □□□□□ □□□□□□□□□□□.

Common mistake / precaution: Every recursive path must reach a base case; otherwise stack exhaustion can occur.

Module 4 revision cue: Re-read official major topics, redraw one key diagram/flow and explain one practical or numerical method without looking at notes.

FORMULA AND PROCEDURE BANK

Rapid Recall Bank

Recall 1

State symbols and SI units before substitution.

Recall 2

Draw the circuit, system boundary, vector or process flow before reasoning.

Recall 3

> For laboratory work: aim → apparatus → diagram → procedure → observation → calculation → result → precautions.

Recall 4

> For a graph: label axes, units, scale, operating region and conclusion.

Recall 5

> For a comparison: use construction, working, result, advantage and limitation.

Recall 6

> For an extended answer: definition/principle, labelled diagram, working steps, application and a caution/limitation.

DIAGRAM LIBRARY

Diagram Library for Rapid Revision

Module 1: Problem Solving and C Foundations

Use the concept flow to revise observation, principle, method and verification.

Module 2: Expressions, Decisions and Loops

Use the concept flow to revise observation, principle, method and verification.

Module 3: Arrays, Strings and Structured Data

Use the concept flow to revise observation, principle, method and verification.

Module 4: Functions, Pointers and Program Design

Use the concept flow to revise observation, principle, method and verification.

SELF-LEARNING

Official Self-Learning and Student Activities

Structured activity

Write algorithm, flowchart and C program for an engineering-unit conversion with validation.

Keep evidence in your record: date, source/observation, diagram/table, key learning and questions for discussion.

Structured activity

Implement and test selection/loop programs using boundary-value cases.

Keep evidence in your record: date, source/observation, diagram/table, key learning and questions for discussion.

Structured activity

Create an array/string program with documented size limits and test data.

Keep evidence in your record: date, source/observation, diagram/table, key learning and questions for discussion.

Structured activity

Develop a menu-driven C program organised into functions; include a small pointer or structure use case.

Keep evidence in your record: date, source/observation, diagram/table, key learning and questions for discussion.

Activity discipline: The supplied PDF assigns the listed self-learning hours. This handbook preserves the activity direction; the teacher’s approved schedule and safety

instructions govern execution.

EXAMINATION READINESS

Assessment Methodology and Examination Pattern

Official assessment structure		
Component	**Official mark / conversion**	**Student evidence**
Attendance	5 marks	End-of-semester attendance component.
Self-learning activities	15 marks	Module-linked activities.
Series examinations	20 + 20 converted	Two 50-mark series examinations.
CIE total	Theory CIE 40	Continuous internal evaluation.
Written ESE	Theory ESE 60	2.5 hours; official Part A / Part B / Part C pattern.

Series-test pattern

The supplied theory pattern uses Part A (1-mark), Part B (3-mark with choice) and Part C (7-mark) distribution across the stated modules. Practical courses follow the supplied practical-test scheme.

Examination evidence

Show a complete method, correct units/conditions, clear diagrams or tables, a result/conclusion and safety awareness for any apparatus or process involved.

EXAM PRACTICE

Module-wise Questions with Answers

Module 1 — Problem Solving and C Foundations

Explain Algorithms, flowcharts and pseudocode and state one correct method, application or precaution.–

An algorithm is a finite, ordered solution plan. Flowcharts show control visually; pseudocode expresses the logic in structured natural-language style. All help detect logic errors before code is written.

Answer framework: State inputs, outputs, constraints and steps; trace one sample input by hand before converting the plan to C.

Important point: Do not start coding before defining what result the program must produce and how invalid input is handled.

Explain C program structure and data and state one correct method, application or precaution.−

A C program uses headers, a main function, declarations, statements and library calls. Variables have types that determine storage, range and permitted operations.

Answer framework: Write a minimal compilable program, declare variables before use, choose types intentionally and use readable names/comments.

Important point: Compiler warnings are information, not decoration; investigate them before trusting output.

Explain Input/output and formatted data and state one correct method, application or precaution.−

Functions such as printf and scanf support interaction, but format specifiers must match variable types. Input validation and predictable output make programs testable.

Answer framework: Specify expected input, use correct format strings, check whether input succeeded where appropriate and print units/labels clearly.

Important point: Never use unsafe or unbounded input patterns; protect array and buffer limits.

Module 2 — Expressions, Decisions and Loops

Explain Operators and expressions and state one correct method, application or precaution.−

Expressions combine operands and operators to compute a value. Precedence, associativity and type conversion affect evaluation and can produce unexpected results.

Answer framework: Parenthesise non-obvious expressions, use intermediate variables for clarity and test boundary values.

Important point: Do not confuse assignment = with equality comparison ==.

Explain Selection statements and state one correct method, application or precaution.–

if/else chooses between conditions; switch selects among discrete cases. Good conditions represent business/engineering rules clearly and include sensible default handling.

Answer framework: Write a truth table or decision table first, test each branch including boundary/default cases and keep nesting understandable.

Important point: A missing break in a switch may cause unintended fall-through unless it is deliberately documented.

Explain Iteration and state one correct method, application or precaution.–

for, while and do-while repeat work under different conditions. Correct initialisation, condition and update guarantee progress toward termination.

Answer framework: State loop invariant, initialise deliberately, trace first/last iteration and test zero/one/maximum-size cases.

Important point: Avoid accidental infinite loops and off-by-one indexing errors.

Module 3 — Arrays, Strings and Structured Data

Explain Arrays and indexing and state one correct method, application or precaution.–

An array stores same-type values in indexed order. C indexes start at zero, so valid element positions are determined by declared size.

Answer framework: Define size/meaning of each index, initialise before read, trace bounds and test first/last element cases.

Important point: Out-of-bounds access is undefined behaviour and can silently corrupt a program.

Explain Strings and state one correct method, application or precaution.–

A C string is a character array terminated by a null character. Length, comparison and concatenation require capacity management and correct termination.

Answer framework: Allocate space including the terminator, state maximum input length and use bounded operations where available.

Important point: Do not assume a character array is a valid string until it is correctly terminated.

Explain Structures and enums and state one correct method, application or precaution.–

Structures group related fields of different types; enumerations give meaningful names to a fixed set of integral states. They improve modelling and readability.

Answer framework: Design a record from the real-world entity, declare fields with appropriate types and access them through clear names.

Important point: Keep structure initialisation and field validation explicit; uninitialised fields create unreliable behaviour.

Module 4 — Functions, Pointers and Program Design

Explain Functions and modular design and state one correct method, application or precaution.–

Functions package a focused task with parameters and a result, reducing duplication and improving testability. Scope determines where variables can be used.

Answer framework: Write a prototype, specify input/output/side effects, test the function with normal and boundary cases and keep it single-purpose.

Important point: Do not rely on hidden global state when parameters/return values express the design more clearly.

Explain Pointers and addresses and state one correct method, application or precaution.–

A pointer stores an address and allows indirect access to an object. Pointer use supports call-by-address, array traversal and dynamic structures but requires careful validity rules.

Answer framework: Draw memory boxes/arrows, initialise pointers, check validity before dereference and pass array length with array parameters.

Important point: Never dereference an uninitialised, null or dangling pointer.

Explain Recursion and storage duration and state one correct method, application or precaution.−

Recursion solves a problem by reducing it to smaller cases with a base case. Storage classes/scope/lifetime affect where variables exist and how long they retain values.

Answer framework: Define base case and reduction step, trace a small call stack and compare automatic/local versus static/global behaviour.

Important point: Every recursive path must reach a base case; otherwise stack exhaustion can occur.

PRACTICE ONLY

Practice Model Paper

Important: This practice paper is based on the official pattern in the supplied syllabus. It is **not** an official SITTTR model question paper.

Part A — short response

1. State one key definition from Module 1: Problem Solving and C Foundations.
2. Identify one diagram, observation, formula or safety condition relevant to Module 1.
3. State one key definition from Module 2: Expressions, Decisions and Loops.
4. Identify one diagram, observation, formula or safety condition relevant to Module 2.
5. State one key definition from Module 3: Arrays, Strings and Structured Data.
6. Identify one diagram, observation, formula or safety condition relevant to Module 3.
7. State one key definition from Module 4: Functions, Pointers and Program Design.
8. Identify one diagram, observation, formula or safety condition relevant to Module 4.

Part B — structured explanation

1. Explain a selected procedure/principle from Module 1 with a labelled diagram, table or method.
2. Explain a selected procedure/principle from Module 2 with a labelled diagram, table or method.
3. Explain a selected procedure/principle from Module 3 with a labelled diagram, table or method.
4. Explain a selected procedure/principle from Module 4 with a labelled diagram, table or method.

Part C — extended response / practical task

1. Develop a complete answer or practical plan for: Problem analysis, algorithms, flowcharts, pseudocode, translators, C program structure, tokens, variables, types and input/output..
2. Develop a complete answer or practical plan for: Arithmetic/relational/logical/bitwise operators, precedence, type conversion, if/switch and for/while/do-while loops..
3. Develop a complete answer or practical plan for: One-/two-dimensional arrays, strings, indexing, standard string operations, structures and enumerations..
4. Develop a complete answer or practical plan for: Library/user functions, scope, parameters, return values, call by value/address, recursion, storage classes and pointers..

LAST-MILE REVISION

Quick Revision

Module 1: Problem Solving and C Foundations

Problem analysis, algorithms, flowcharts, pseudocode, translators, C program structure, tokens, variables, types and input/output.

- Match each topic to CO1.
- Redraw or rehearse one key method.
- Check one common mistake/precaution.

Module 2: Expressions, Decisions and Loops

Arithmetic/relational/logical/bitwise operators, precedence, type conversion, if/switch and for/while/do-while loops.

- Match each topic to CO2.
- Redraw or rehearse one key method.
- Check one common mistake/precaution.

Module 3: Arrays, Strings and Structured Data

One-/two-dimensional arrays, strings, indexing, standard string operations, structures and enumerations.

- Match each topic to CO3.
- Redraw or rehearse one key method.

- Check one common mistake/precaution.

Module 4: Functions, Pointers and Program Design

Library/user functions, scope, parameters, return values, call by value/address, recursion, storage classes and pointers.

- Match each topic to CO4.
- Redraw or rehearse one key method.
- Check one common mistake/precaution.

Common mistakes: Writing an unlabelled diagram; ignoring units, polarity or conditions; treating a practical result as valid without observations; confusing a definition with an application; and omitting a safety precaution where the topic uses apparatus, current, heat, chemicals or tools.

Exam checklist: Read the command word; answer every part; draw clean labelled diagrams; use a clear calculation/procedure; state result with units or observation; and reserve two minutes for checking.

VOCABULARY

Glossary

Important terms from the syllabus	
Term / topic	**One-line meaning**
Algorithms, flowcharts and pseudocode	An algorithm is a finite, ordered solution plan.
C program structure and data	A C program uses headers, a main function, declarations, statements and library calls.
Input/output and formatted data	Functions such as printf and scanf support interaction, but format specifiers must match variable types.
Operators and expressions	Expressions combine operands and operators to compute a value.
Selection statements	if/else chooses between conditions; switch selects among discrete cases.
Iteration	for, while and do-while repeat work under different conditions.
Arrays and indexing	An array stores same-type values in indexed order.
Strings	A C string is a character array terminated by a null character.
Structures and enums	Structures group related fields of different types; enumerations give meaningful names to a fixed set of integral states.
Functions and modular design	Functions package a focused task with parameters and a result, reducing duplication and improving testability.
Pointers and addresses	A pointer stores an address and allows indirect access to an object.
Recursion and storage duration	Recursion solves a problem by reducing it to smaller cases with a base case.

LEARNING RESOURCES

References

Recommended C-programming references

1. E. Balagurusamy, Programming in ANSI C.
2. Brian W. Kernighan and Dennis M. Ritchie, The C Programming Language.
3. Yashavant Kanetkar, Let Us C.

Approved public C-programming sources

-
-

The listed public eplace an official detailed syllabus.